

Overview of Q-learning

Subject: Q-learning

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

1.

$\alpha \gamma \max_a Q(S_{t+1}, a)$: the maximum reward that can be obtained from state S_{t+1} (weighted by learning rate and discount factor) An episode of the algorithm ends when state S_{t+1} is a final or terminal state. However, Q-learning can also learn in non-episodic tasks (as a result of the property of convergent infinite series). If the discount factor is lower than 1, the action values are finite even if the problem can contain infinite loops or paths. For all final states s_f , $Q(s_f, a)$ is never updated, but is set to the reward value r observed for state s_f . In most cases, $Q(s_f, a)$ can be taken to equal zero.

2.

Double Q-learning Because the future maximum approximated action value in Q-learning is evaluated using the same Q function as in current action selection policy, in noisy environments Q-learning can sometimes overestimate the action values, slowing the learning. A variant called Double Q-learning was proposed to correct this. Double Q-learning is an off-policy reinforcement learning algorithm, where a different policy is used for value evaluation than what is used to select the next action. In practice, two separate value functions Q_A and Q_B are trained in a mutually symmetric fashion using separate experiences. The double Q-learning update step is then as follows:

3.

Initial conditions (Q0) Since Q-learning is an iterative algorithm, it implicitly assumes an initial condition before the first update occurs. High initial values, also known as "optimistic initial conditions", can encourage exploration: no matter what action is selected, the update rule will cause it to have lower values than the other alternative, thus increasing their choice probability. The first reward r can be used to reset the initial conditions. According to this idea, the first time an action is taken the reward is used to set the value of Q . This allows immediate learning in case of fixed deterministic rewards. A model that incorporates reset of initial conditions (RIC) is expected to predict participants' behavior better than a model that assumes any arbitrary initial condition (AIC). RIC seems to be consistent with human behaviour in repeated binary choice experiments.

4.

Multi-agent learning Q-learning has been proposed in the multi-agent setting (see Section 4.1.2 in). One approach consists in pretending the environment is passive. Littman proposes the minimax Q learning algorithm.

5.

Reinforcement learning involves an agent, a set of states S , and a set A of actions per state. By performing an action $a \in A$, the agent transitions from state to state. Executing an action in a specific state provides the agent with a reward (a numerical score). The goal of the agent is to maximize its total reward. It does this by adding the maximum reward attainable from future states to the reward for achieving its current state, effectively influencing the current action by the potential future reward. This potential reward is a weighted sum of expected values of the rewards of all future steps starting from the current state. As an example, consider the process of boarding a train, in which the reward is measured by the negative of the total time spent boarding (alternatively, the cost of boarding the train is equal to the boarding time). One strategy is to enter the train door as soon as they open, minimizing the initial wait time for yourself. If the train is crowded, however, then you will have a slow entry after the initial action of entering the door as people are fighting you to depart the train as you attempt to board. The total boarding time, or cost, is then:

6.

$$Q_{t+1}^A(s_t, a_t) = Q_t^A(s_t, a_t) + \alpha_t(s_t, a_t) (r_t + \gamma \max_a Q_t^B(s_{t+1}, a) - Q_t^A(s_t, a_t))$$

$$Q_{t+1}^B(s_{t+1}, a) = Q_t^B(s_{t+1}, a) + \alpha_t(s_{t+1}, a) (r_{t+1} + \gamma \max_a Q_{t+1}^A(s_{t+1}, a) - Q_t^B(s_{t+1}, a))$$
, and

7.

In state s perform action a ; Receive consequence state s' ; Compute state evaluation $v(s')$; Update crossbar value $w'(a, s) = w(a, s) + v(s')$. The term "secondary reinforcement" is borrowed from animal learning theory, to model state values via backpropagation: the state value $v(s')$ of the consequence situation is backpropagated to the previously encountered situations. CAA computes state values vertically and actions horizontally (the "crossbar"). Demonstration graphs showing delayed reinforcement learning contained states (desirable, undesirable, and neutral states), which were computed by the state evaluation function. This learning system was a forerunner of the Q-learning algorithm. In 2014, Google DeepMind patented an application of Q-learning to deep learning, entitled "deep reinforcement learning" or "deep Q-learning," that can play Atari 2600 games at expert human levels.

8.

$$Q^{new}(S_t, A_t) \leftarrow (1 - \alpha \text{ learning rate}) \cdot Q(S_t, A_t) + \alpha \text{ learning rate} \cdot (R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t))$$

$$Q^{new}(S_t, A_t) \leftarrow (1 - \alpha \text{ learning rate}) \cdot Q(S_t, A_t) + \alpha \text{ learning rate} \cdot (R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t))$$

$$\{R_{t+1}\} + \underbrace{\gamma}_{\text{discount factor}} \cdot \underbrace{\max_a Q(S_{t+1}, a)}_{\text{estimate of optimal future value}} - \underbrace{V_t}_{\text{new value (temporal difference target)}}$$